

Comparison of Deep Reinforcement Learning Methods for Safe and Efficient Autonomous Vehicles at Pedestrian Crossings

Alexandre Brunoud*, Alexandre Lombard†, Meng Zhang‡, Abdeljalil Abbas-Turki§, Nicolas Gaud¶, Abder Koukam||

CIAD UMR 7533, Univ. Bourgogne Franche-Comté, UTBM, F-90010 Belfort, France

Email: *alexandre.brunoud@utbm.fr, †alexandre.lombard@utbm.fr,

‡meng.zhang@utbm.fr, §abdeljalil.abbas-turki@utbm.fr, ¶nicolas.gaud@utbm.fr, ||abder.koukam@utbm.fr

Abstract—These past years, the domain of Connected and Autonomous Vehicles (CAV) has been extremely flourishing, with fully autonomous self-driving cars being an active research area. The most challenging aspect of the driving task is the interaction between the driver agent and the environment, especially to handle conflicts with the other road users. The most vulnerable ones are the pedestrians. To ensure better safety at crosswalks, this paper focuses on planning a vehicle trajectory, to pass without risks, while cooperating with the pedestrian. According to recent works, Deep Reinforcement Learning can be used to efficiently control the vehicle in a complex environment. This paper assesses both widely held continuous action space approaches for controlling the vehicle: Deep Deterministic Policy Gradient and Proximal Policy Optimization. Both approaches are used to allow the vehicle to safely and efficiently interact with a pedestrian willing to cross the road at the crosswalk. The main contributions of the paper are the design and parameterization of these approaches for this single conflict use-case, the feasibility study of both approaches, and the detailed comparison showing their benefits and limitations.

I. INTRODUCTION

More and more cars are being equipped with Advanced Driver Assistance Systems (ADAS), which are used to give to the vehicles and their drivers more data about their environment, and eventually to allow the vehicle to dynamically react and adapt to changes in the environment. The long-term goal is the design of a fully-autonomous self-driving car, yet this task is still in progress because of the complexity of the decision process. A challenging aspect of driving is the interaction between the vehicles and other road users, notably the pedestrians. Eventually, the capacities of autonomous vehicles could be used to improve the safety of pedestrians, thanks to an automated detection and an adapted speed profile.

Here, the focus is on the vehicle-pedestrian conflict at a crosswalk, which is the most common situation. In the last decade, several works have focused on the signalization system that displays the vehicle's intention, for the single conflict interaction use-case. More precisely, a color, a gaze, or a smile are displayed on the front of the vehicle [17]. A projected signal on the road was also considered [5].

In addition to these studies, it is commonly admitted [2], [10] that the pedestrian takes the decision according to the position and the speed of the vehicle. In [20], [21], the authors show that the speed profile of the vehicle can contribute to improve the pedestrian crossing performance, using Pontryagin's maximum principle.

In this paper, the target goal is to define the optimal vehicle trajectory, preserving the pedestrian safety while maximizing the overall speed of the vehicle. The crux issue that hinders the achievement of this objective is the randomness of pedestrian behavior, which makes the environment uncertain. In practice, the optimal control of the vehicle needs at least to estimate (1) the safety margin of the pedestrian to invite her/him to cross and (2) her/his exit time to allow the vehicle to be ready to exit the soonest. Both pedestrian safety margin and exit time are not deterministic. They depend on many factors, such as pedestrian gender, age, social community, and so on [11]. Moreover, even with known values, the pedestrian may encounter unforeseeable events such as falling or hesitation, which may force the vehicle to come to emergency braking for avoiding or mitigating the collision.

In order to overcome such obstacles, this paper relies on a suitable design of vehicle control based on deep reinforcement learning. The vehicle is trained according to several random pedestrian crossing profiles (including falls, slow speed...), in order to simultaneously maintain the safety distance and optimize the performance of the interaction. There exists exact solutions such as quadratic programming (QP) who adapt the vehicle speed to a particular trajectory shape. This kind of solutions is weakly responsive to environment changes, notably because of the computation times. To address this issue, we use Deep Reinforcement Learning (DRL) which is a subfield of machine learning [3]. DRL is continuously learning from experience. It has shown good results in recent years for complex problems, and has the advantage of learning with long-term rewards.

Our model has a continuous action-space, which limits the choices of the algorithm. The Deep Deterministic Policy Gradient algorithm (DDPG) has been successfully used on

this problem [22], showing the relevance of communicative behavior to ensure safety and efficiency in crosswalk. The Proximal Policy Gradient algorithm (PPO) is also a good candidate for this problem. Indeed, it is sample efficient compare with other policy gradient algorithms, and it has a good exploration technique. DDPG and PPO are the most commonly used DRL methods for continuous action space. Due to the DRL structure, the optimal strategy is extremely dependent on the reward function for both algorithms. Therefore, the simulation part is meticulous, requiring a lot of experiments.

In this paper, we define a single vehicle-pedestrian conflict environment, and compare on it the two previously mentioned algorithms: DDPG and PPO. The objective is multiple: defining key properties for the design of DRL strategies for this problem, assessing the feasibility of both approaches for the problem of optimization of pedestrian crossings, and giving a detailed comparison of DDPG and PPO for this specific use-case.

II. USE-CASE: SIMULATION OF THE INTERACTION

We assume a 1-way street where two agents will interact, one is a pedestrian and the other is our CAV. Both wants to cross the street. The crosswalk direction and road direction are straight lines and are perpendicular, as illustrated in Fig. 1. Upon the detection of a pedestrian coming to the

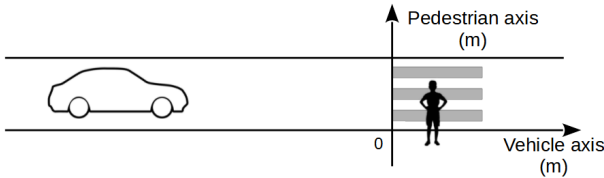


Fig. 1. Scheme of the positions of the vehicle and the pedestrian.

crosswalk, the car will have to react to this new agent. In the present scenario, the action over the steering wheel is neglected, and the car has to follow the road: the considered movement is 1D. This restriction on the action-space is designed to focus on finding a cooperation between the two agents related to the vehicle speed. If the vehicle was allowed to turn, then it would only dodge the pedestrian to prevent an accident, falling back to a classical path planning problem.

Once the pedestrian detected, the car must only adapt its acceleration, which is commonly used by the driver thanks to the accelerator and the brake pedals [1]. The simulation ends some time after the pedestrian has finished crossing, to let the car pass and resume its speed.

A. Pedestrian Model

In the present model, the pedestrian is always willing to cross the street. He is walking towards the crosswalk with a constant speed corresponding to his classic walking speed. Once arrived at the crosswalk, he stops to see if he can cross safely. The decision to cross or not is made according to the Critical Gap (CG) theory [10]. It assumes that it exists, for

every pedestrian, a given amount used as threshold to accept or reject a temporal gap (time before vehicle arrives). The amount depends on an estimation of the pedestrian crossing and is compared with the vehicle gap. If the CG is higher, the pedestrian crosses, otherwise she/he waits. A reaction time is considered.

This CG factor depends on the pedestrian characteristics, such as age and gender, with the same parameter values as in [8]. For example, an elderly person may accept a shorter CG to cross the road, as displayed in Fig. 2. The black line represents the estimation of the pedestrian crossing, depending on his speed and the crosswalk length.

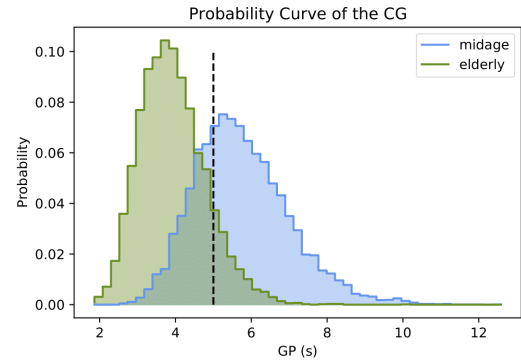


Fig. 2. CG Distribution for 2 age groups (50 bins)

The car arrival time is computed with its speed and its position, hence we can act on this value by controlling the acceleration of the CAV. Therefore, if the vehicle slows down, the pedestrian will be more likely to cross. Nevertheless, this reaction to the car movement rely on the pedestrian estimation and cooperation. The pedestrian can make a unilateral decision: force his way through the crosswalk or let the car pass even though he has a large and sufficient CG. To increase the realism of the simulation, as well as to generate borderline cases for the evaluation of the policy, we consider these aspects in our model.

For the present model, the pedestrian speed changes according to his position. The speed is considered constant before the crosswalk, as well as after the crosswalk. However, the pedestrian behaves differently during the crossing. The model illustrated in Fig. 3, gives the general simulated shape. In this model, the pedestrian can have a different desired crossing speed. The model is derived from the observed data of real testers that are provided in [19]. In addition, the simulated pedestrian may stop randomly in the crosswalk, to consider falls or hesitations.

B. Markov Decision Process

The car needs to take action in real-time depending on the analyzed environment. To this end, it must analyze the environment, make decisions from its analysis and finally act accordingly. A computation time to reiterate these steps is considered. This will lead to a new state of the environment

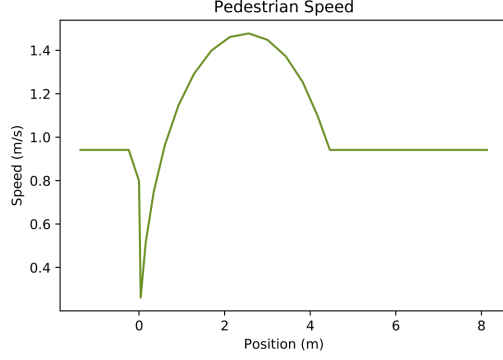


Fig. 3. Pedestrian speed in relation to his position.

due to exterior changes (pedestrian agent) and the actions already taken by the car, at each new time step.

Moreover, it can be assumed that the new state depends only on the current state and the last achieved action, thus, respecting the Markov property. Here, the car may adapt its acceleration to the current situation only if we add the acceleration (previous action) in our state.

Finally, according to the properties stated above, the present system can be modeled as a Markov Decision Process (MDP) defined by four elements: (S, A, P, R) where S is the state space, A the action space, P the transition probabilities and R the immediate reward. This makes most RL approaches usable on this system [15].

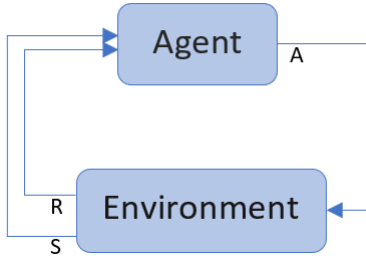


Fig. 4. Reinforcement Learning Diagram

C. Action and State Parameters

As our system is focused on the control of the velocity of the CAV, the action space can be limited to the control of the acceleration of the car, i.e. a single real-valued variable. Its value interval is defined in the Table I.

For the state space, the parameters are also described in the Table I. The movement parameters of the car can be computed thanks to the geographic coordinates (GPS). It is to be noted that we are using the previous action in our state, as mentioned previously. Adding the previous action in our state is done so that the derivative of the action can affect the current action. Apart from the car acceleration and the pedestrian speed, parameters are not hardly constraint by their intervals, but are restricted to it due to the reward.

The pedestrian speed and position are estimated thanks to optical sensors (Camera). The time is extracted using a clock. Those first parameters are the classical state of the environment. Unlike [16], this information is supposed to be always available.

For the purpose of helping the algorithm to converge faster and to reduce the calculation difficulty, secondary parameters linked to the reward are used. Thus, pedestrian presences, safety factor and speed difference are added to the state space parameters.

TABLE I
LIST STATE PARAMETERS

Name	Value interval	Initial Value	Units
Car Acceleration	$[-4.0; 2.0]$	0.0	m/s^2
Car Speed	$[0.0; 11.0]$	$[9.0; 11.0]$	m/s
Car Position	$[-32.5; 60.0]$	$[-32.5; -12.5]$	m
Pedestrian Speed	$[0.0; 2.5]$	$[0.9; 1.1]$	m/s
Pedestrian Position	$[-2.0; 10.0]$	$[-2.0; -1.0]$	m
Safety Factor *	$[-27.0; 32.5]$	$[-14.0; 26]$	m
Pedestrian Presence in Crosswalk	0, 1	0	$\emptyset$
Pedestrian Presence After Crosswalk	0, 1	0	$\emptyset$
Speed Difference	$[-11.0; 0.0]$	0.0	m/s
Time Spending	$[0.0; 30.0]$	0.0	s

* : Compute wrt other parameters

D. Reward Parameters

Defining the reward is the most challenging part of the design of an RL strategy. To compare DDPG and PPO, we train them both on the same environment and therefore using the same reward. According to the literature overview and the target behavior to be induced, it can be divided into four parts:

- The first and main part is the safety reward. It is used to prevent any accident. The car is heavily penalized if it causes an accident, i.e. if the car touches the pedestrian. Indeed, preventing accidents is mandatory for building a safe CAV. In this model, the penalization does not depend on the car speed because we want to limit the risk of accident at any level of dangerousness. The car is also incited to respect safety distance. If the car is too close from the crosswalk, it is supposed to pass. Otherwise, the pedestrian has priority.
- The second part is urging the car to keep as much speed as possible while respecting the speed limits and the traffic rules such as the direction of travel and speed limit. To enhance this reward, the car speed will adopt a behavior of reducing speed until the pedestrian has time to cross, then accelerating to the speed limit.
- The third part aims at smoothing the acceleration curve over time by penalizing the changes of action.
- The last part is a mix of rewards which are less important for the car trajectory. It is preventing certain

behaviors, especially at the end of the episode. It urges the car to accelerate when it should: after the pedestrian crossing if it has the time to stop, or before otherwise. It also penalizes negative speed.

TABLE II
LIST REWARD PARTS

Name	Description	Variables
Safety	Pedestrian safety assessment Preventing accidents	Initial car position Car position Pedestrian position Safety factor
Speed	Speed conservation Avoid time wasting	Initial car speed Initial pedestrian speed Car speed Pedestrian position Previous speed
Acceleration	Acceleration conservation	Previous acceleration Current acceleration
End road	Support good behavior	Initial car speed Car speed Pedestrian Position Car position Previous car position

III. OVERVIEW OF THE ALGORITHMS

The first idea to treat continuous action-space problem is the discretization of the action space, to be able to use DQN [9]. This method however is not efficient because of the curse of dimensionality and the accuracy of the solution. A review of the literature on continuous action space MDP leads us to consider two approaches (PPO and DDPG) which are described thereafter.

A. Definitions

We consider an MDP (S, A, P, R) . Following usual definitions:

- π is the actor policy of the current algorithm, either deterministic ($\pi_d : S \rightarrow A$ defining the action $a \in A$ to be applied when in state $s \in S$), or stochastic ($\pi_s : S \times A \rightarrow [0, 1]$ defining the probability to perform the action a in the state s).
- γ is the discounting factor, a constant in $[0, 1]$
- $V_\pi : S \rightarrow \mathbb{R}$ is the value function is the expected return starting from state $s \in S$:

$$V_\pi(s) = E_\pi \left[\sum_{t=0}^{T-1} \gamma^t r_t \mid s_t = s \right]$$

- $Q_\pi : S \times A \rightarrow \mathbb{R}$ is the action-value function is the expected return starting from state $s \in S$ and taking action $a \in A$:

$$Q_\pi(s, a) = E_\pi \left[\sum_{t=0}^{T-1} \gamma^t r_t \mid s_t = s, a_t = a \right]$$

- $A_\pi : S \times A \rightarrow \mathbb{R}$ is the advantage function, i.e. the difference between the two previous elements. It represents the advantage got from the action a .

$$A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$$

- θ and θ^Q are the policy and critic networks weights. (θ^Q is only used for the DDPG)

B. Deep Deterministic Policy Gradient

The Deep Deterministic Policy Gradient (DDPG) is a successor of the DQN model in continuous action-space. DDPG is a model-free, off-policy actor-critic algorithm [6]. This algorithm has two separate parts:

- The first part is a deterministic policy standing for the actor. The advantage of this kind of policy have already been studied in deterministic policy gradient methods (DPG) [14]. It's easier to compute, and it allows a link between the Q-value and the policy optimization we will be using.
- The second part is the Q-value, standing for the critic. This part is judging the policy action to compare with the results.

The main idea of this algorithm is the merge between actor-critic method from Bellman equation and the characteristics of DPG. From the Bellman equation (1):

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \tilde{R}_{s_{t+1}}] \quad (1)$$

where

$$\tilde{R}_{s_{t+1}} = \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})] \quad (2)$$

The critic can be approximated by minimizing the loss (3).

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^{\pi_n}, a_t \sim \pi_n, r_t \sim E} [(Q(s_t, a_t \mid \theta^Q) - y_t)^2] \quad (3)$$

where

$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \pi_{target}(s_{t+1}) \mid \theta^Q) \quad (4)$$

π is updated thanks to the chain-rule between the policy and the Q-value.

This algorithm uses some tools to work:

- **Replay Buffer:** steps are stored in a buffer, and training is performed on a sample of this buffer. It reduces the correlations in the batch, thus the stationary. Moreover, the reuse of steps makes DDPG more sample efficient.
- **Soft-target updates π_{target} and Q_{target} :** using a target network for the critic and for the actor prevents divergence and instability. DDPG keep a consistent target this way, even if it is slower to converge.
- **Exploration parameter π_n :** A noise is added to the policy result. This noise is a Gaussian or an Ornstein-Uhlenbeck process and is treating the problem of exploration.

TABLE III
DDPG HYPERPARAMETERS FOR OUR MODEL

Name	Value
Time step (dt)	0.3
Gamma(γ)	0.99
Tau (Soft-updates)	5e-3
Layers Actor Size	(10,64,64,1)
Layers Critic Size (state part)	(10,48,48)
Layers Critic Size (action part)	(1,48)
Layers Critic Size (merge part)	(96,48,1)
Learning rate Actor	3e-3
Learning rate Critic	3e-4
Size Replay Buffer	1e6
Start standard deviation noise	0.5
End standard deviation noise	0.1
Episodes Number	2e6

C. Proximal Policy Optimization

The Proximal Policy Optimization algorithm (PPO) [13] is the successor of the Trust Region Policy Optimization (TRPO) [12]. It is based on alternation between sampling and optimization of a “surrogate” objective (improving the current stochastic policy). The classical policy gradient method is used with a stochastic gradient ascent on:

$$\hat{g} = \hat{\mathbb{E}}_t[\nabla_{\theta} \log(\pi(a_t | s_t)) \hat{A}_t] \quad (5)$$

TRPO adds a constraint on the policy update $\hat{g}$ using Kullback-Leibler divergence. PPO transforms the constraint into a penalization with a scaling parameter. Due to the risk of destructively large updates of the policy, they are clipped, as illustrated in loss (6).

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)] \quad (6)$$

TABLE IV
PPO HYPERPARAMETERS FOR OUR MODEL

Name	Value
Time Step (dt)	0.3
Gamma (γ)	0.99
Layers Policy and Critic Size	(10,64,64,1)
Learning Rate Actor	3e-4
Learning Rate Critic	3e-4
Clip Parameter	0.2
Variance Gaussian Noise	0.5
Batch Size	2000
Episodes Number	1000

IV. EVALUATION

A. Simulation

This section presents the technical tools used to simulate the environment, and the application of the two models on it. OpenAI Gym [4] is a toolkit made for reinforcement learning model, our custom environment implements the GymEnv interface. Python is a programming language that contains many libraries, and especially PyTorch. We implement the two models with it. Jupyter Notebook is the computing platform.

First tests were restricted to a certain interval positions, which forces the vehicle to let the pedestrian passes. In the final test, the vehicle must choose the good trajectory, which only depends on the initial position and speed. Hyperparameters are set thanks to [6] and [13]. The reward parameters are defined by training our models and analyzing returns such as the reward diagrams. 1000-episode sample are used to evaluate the models.

The implementation part is available at the following link: https://github.com/BrunoudA/ComparisonDDPG_PPO.

B. Reward results

The two algorithms run on 2 millions steps. The DDPG and PPO rewards are quickly converging in Fig. 5. Although, the DDPG reward is very unstable, both are converging, in less than $2 * 10^5$ steps for DDPG and $8 * 10^5$ steps for PPO. The final PPO and DDPG average step reward are very similar, but the PPO training is three times faster.

In Fig. 6, we run the algorithms over one hour. PPO uses twice as many steps. Moreover, it is getting a better final average step reward.

These results are related to the core structure of those algorithms. DDPG updates the neural network weights more often on small batch (extracted from RB), which makes the computation more time-consuming. Whereas PPO updates on big batch directly taken from the current policy, which makes the computation more step-consuming. It is important to note that we tried many rewards and the two algorithms may react differently with those modifications. For example, PPO adapts more easily to sparse rewards in this use-case.



Fig. 5. Rewards Diagram in relation to steps

C. Speed results

In the next diagrams, the red line represents the initial car speed as well as the limit speed. The blue line represents the average speed and acceleration, and the blur is computed by adding the standard deviation. Different scenarios are analyzed.

a) *Pedestrian waiting status for PPO*: In Fig. 7, the algorithm quickly converges to the correct speed during the episode. The vehicle speed oscillates around the limit speed. Keeping the same speed indicates that the pedestrian should not cross, so the vehicle has the expected reaction.

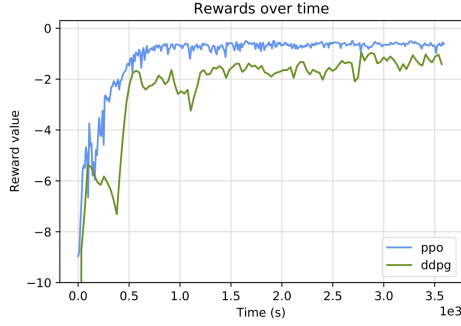


Fig. 6. Rewards Diagram in relation to time

We notice a slight acceleration before the crosswalk and a deceleration after. The speed variance is small, thus PPO behavior is very stable. High oscillations at the start of the episode can be explained by the range of the initial state space. But the slow-down at the start of the episode is not expected in this scenario.

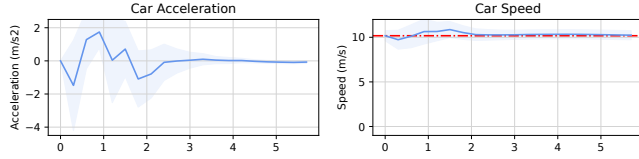


Fig. 7. Car Acceleration and Car Speed versus time curves for PPO (pedestrian waits)

b) Pedestrian crossing status for PPO: In Fig. 8, we can separate the episode in four steps. The vehicle brakes in the first part to let the pedestrian crosses. Then it keeps a rather constant and small acceleration, thus our speed increase slowly. In the third part, the car accelerate to pass the crosswalk with an acceleration peak. We want to finish the episode with a speed close to the limit speed. As expected, the speed converges to the limit speed with a small acceleration in the last part.

There are two problems about the speed diagram. The waves in the speed diagram may lead to misinterpretation from the pedestrian (the peaks in the second part). And the transition between the brake and the acceleration parts is a bit sharp. The variance is small except when the vehicle accelerate, because it depends on the pedestrian behavior: when he will leave the crosswalk, the car will be able to pass. It is also linked to the initial car position. Indeed, if it was far to the crosswalk, then it will accelerate faster because his position is safer.

c) Pedestrian waiting status for DDPG: In Fig. 9, the acceleration is decreasing before the crosswalk and is staying constant afterwards. Thus, the vehicle speed is too slow at the end of the episode. We observe that the variance stays high throughout the episode, though it should decrease over time. We computed DDPG multiple times, the final acceleration value varies with the training data.

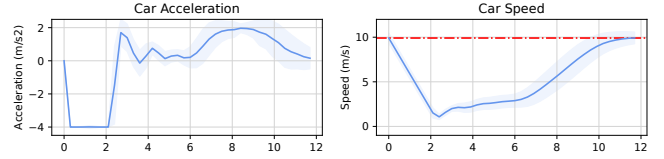


Fig. 8. Car Acceleration and Car Speed versus time curves for PPO (pedestrian crosses)

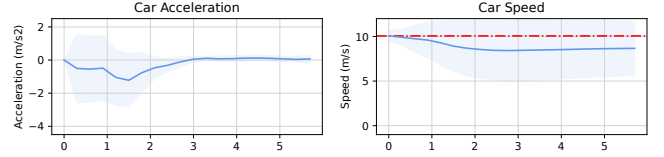


Fig. 9. Car Acceleration and Car Speed versus time curves for DDPG (pedestrian waits)

d) Pedestrian crossing status for DDPG: In Fig. 10, the car brakes to let the pedestrian crosses. Then, the car slowly accelerates during the pedestrian crossing compared with the PPO speed shape. In some episodes, the DDPG acceleration may change rapidly, but in general its curve is smoother than the PPO acceleration. We can see a rather constant variance over the time in the second part. We also look at these parameters over its position, and we see that the variance is decreasing over time. DDPG try to follow the same trajectory near the crosswalk for all vehicle depending on its position. Its acceleration variance is still very high, it will take some time to stabilize.

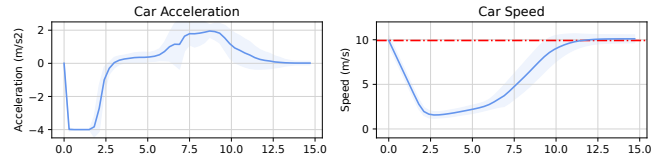


Fig. 10. Car Acceleration and Car Speed versus time curves for DDPG (pedestrian crosses)

D. Emission and exit-time results

TABLE V
MODEL DESCRIPTION

Name	QP	DDPG	PPO
Accident Occurrence	0	0	0
Avg. Car Speed	7.19 m/s	7.38 m/s	7.03 m/s
Avg. Ped. Speed	0.99 m/s	0.99 m/s	1.00 m/s
CO2 Consumption	21636.1 mg	20083.6 mg	22789.3 mg
Avg. Exit Time (Car)	4.55 s	4.49 s	4.89 s
Avg. Exit Time (Ped.)	6.60 s	6.57 s	6.55 s

In terms of speed conservation, PPO is very efficient, but also suffers from some oscillations. Indeed, the average pedestrian speed is also the average initial pedestrian speed. It shows that the pedestrian priority is respected. Due to the shape of the speed curve, PPO has a higher amount of CO2

emission. DDPG is also very efficient, but its speed is not converging to the limit speed.

The pedestrian is more receptive to the PPO actions, as we can see with his average exit time. We try both algorithms on two test samples. One has a greater likelihood that the pedestrian will stop, the other adds randomness in the speed pedestrian curve. We also use initial states with a safety factor closed to zero. Therefore, the car must act in the first steps to define its trajectory.

For QP, we need to adjust a different trajectory for every case when the pedestrian stops, which is very difficult to compute due to the pedestrian unpredictability. The acceleration curve is then much too steep.

We still have some problems with pedestrian stopping right before the end of the crosswalk. It can cause accidents for those three algorithms in this particular situation. To deal with it, we could build a safe mode by increasing the safety reward. However, both algorithms are converging faster with low safety reward. Thus, the best issue seems to be a curriculum-based training strategy [7], [18].

V. CONCLUSION

An environment is defined for the safe pedestrian crossing problem, with a continuous action-space. Then, implementations of two DRL methods (PPO and DDPG) are proposed to solve it. After adjustments of the reward and the algorithm hyperparameters, the two strategies are applied on the problem, ensuring their feasibility and suitability for this specific use-case. According to the performed measures, both PPO and DDPG are able to learn an efficient behavior in terms of speed profile. PPO is giving better reward and faster converging. DDPG has smoother acceleration and speed curves, but higher variance.

Further work is still required to improve the pedestrian safety. One of the main idea is to improve the communication between the vehicle and the pedestrian, for instance by adding a dedicated signaling display on the vehicle. Yet, the impact of such system on the trajectory of the pedestrian is to be studied. In the meantime, the action space of the vehicle would become both continuous (speed profile) and discrete (by the control of the signaling system), which requires further work, for instance by application of parameterized action-space.

REFERENCES

- [1] Claudia Ackermann, Matthias Beggato, Luka-Franziska Bluhm, Alexandra Loew, and Josef Krems. "Deceleration parameters and their applicability as informal communication signal between pedestrians and automated vehicles". *Transportation Research Part F Traffic Psychology and Behaviour*, 62, 03 2019.
- [2] Claudia Ackermann, Matthias Beggato, Luka-Franziska Bluhm, Alexandra Löw, and Josef F Krems. Deceleration parameters and their applicability as informal communication signal between pedestrians and automated vehicles. *Transportation research part F: traffic psychology and behaviour*, 62:757–768, 2019.
- [3] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. "Deep reinforcement learning: A brief survey". *IEEE Signal Processing Magazine*, 34(6):26–38, 2017.
- [4] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. "Openai gym". *CoRR*, abs/1606.01540, 2016.
- [5] Ming Hou, Karthik Mahadevan, Sowmya Somanath, Ehud Sharlin, and Lora Oehlberg. Autonomous vehicle-cyclist interaction: peril and promise. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, pages 1–12, 2020.
- [6] Timothy Lillicrap, Jonathan Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. "Continuous control with deep reinforcement learning". *CoRR*, 09 2015.
- [7] Hailan Ma, Daoyi Dong, Steven X Ding, and Chunlin Chen. Curriculum-based deep reinforcement learning for quantum control. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.
- [8] Charles Manski, Mark Manuszak, and Sanghamitra Das. "Walk or wait? an empirical analysis of street crossing decisions". *Journal of Applied Econometrics*, 20:529–548, 05 2005.
- [9] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei Rusu, Joel Veness, Marc Bellemare, Alex Graves, Martin Riedmiller, Andreas Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumar, Daan Wierstra, Shane Legg, and Demis Hassabis. "Human-level control through deep reinforcement learning". *Nature*, 518:529–33, 02 2015.
- [10] Sn Mohamad Nor, Basil Daniel, Rizati Hamidun, Walid Al Bargi, Munzilah Rohani, Joewono Prasetyo, Mohamad Aman, and Kamarudin Ambak. Analysis of pedestrian gap acceptance and crossing decision in kuala lumpur. *MATEC Web of Conferences*, 103:08014, 01 2017.
- [11] Amir Rasouli and John K Tsotsos. Autonomous vehicles that interact with pedestrians: A survey of theory and practice. *IEEE transactions on intelligent transportation systems*, 21(3):900–918, 2019.
- [12] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. "Trust region policy optimization". *CoRR*, abs/1502.05477, 2015.
- [13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. "Proximal policy optimization algorithms". *CoRR*, abs/1707.06347, 2017.
- [14] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. "Deterministic policy gradient algorithms". *31st International Conference on Machine Learning, ICML 2014*, 1, 06 2014.
- [15] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA, 2018.
- [16] Sarah Thornton. "Autonomous vehicle speed control for navigation of occluded pedestrian crosswalk". *CoRR*, abs/1802.06314, 2018.
- [17] Tram Thi Minh Tran, Callum Parker, and Martin Tomitsch. A review of virtual reality studies on autonomous vehicle–pedestrian interaction. *IEEE Transactions on Human-Machine Systems*, 2021.
- [18] Honghu Xue, Benedikt Hein, Mohamed Bakr, Georg Schilbach, Bengt Abel, and Elmar Rueckert. Using deep reinforcement learning with automatic curriculum learning for mapless navigation in intralogistics. *arXiv preprint arXiv:2202.11512*, 2022.
- [19] Meng Zhang. Project: Cooperative crossing behaviors between cavs and pedestrians., 2021.
- [20] Meng Zhang, Abdeljalil Abbas-Turki, Alexandre Lombard, and Abderrafaa Koukam. Connected and autonomous vehicles cooperate with the pedestrian in industrial sites based on trajectory optimization and vehicle signalization system. In *2020 IEEE Intelligent Vehicles Symposium (IV)*, pages 188–194. IEEE, 2020.
- [21] Meng Zhang, Abdeljalil Abbas-Turki, Alexandre Lombard, Abderrafaa Koukam, and Kang-Hyun Jo. Autonomous vehicle with communicative driving for pedestrian crossing: Trajectory optimization. In *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, pages 1–6. IEEE, 2020.
- [22] Meng Zhang, Abdeljalil Abbas-Turki, Yazan Mualla, Abderrafaa Koukam, Xiaowei Tu, Alexandre Lombard, and Nicolas Gaud. "Coordination between connected automated vehicles and pedestrians at industrial sites based on deep reinforcement learning". *IEEE Transactions on intelligent transportation systems*, Under submission.